

41. Programování v jazyku symbolických instrukcí (činnost počítače, strojový jazyk, symbolický jazyk, assembler)

41. Programování v jazyku symbolických instrukcí

SZZ okruh 41 (FIT BUT). Od činnosti CPU po assembler.

Shrnutí

Činnost počítače

- CPU (ALU, registry, řadič, hodiny) + paměť + I/O, propojeno sběrnicemi.
- **Cyklus: fetch** (IR → [IP]) → aktualizace IP → **decode** → **execute** (skoky mění IP).

Symbolický jazyk a instrukce

- **Strojový kód** (binární, nepřenositelný) → **symbolický jazyk** (čitelné zkratky, ~1:1 mapování) → vyšší jazyky.
- Typy instrukcí: přenosové (MOV/PUSH/POP), aritmetické (ADD/MUL/DIV), logické, posuvy/rotace, **skokové** (JMP, podmíněné dle příznaků), řetězové, řídicí (CALL/RET).
- Registry: EAX/EBX/ECX/EDX, ESP/EBP (zásobník), ESI/EDI (řetězce); **EFLAGS** (CF, ZF, OF, SF, DF).
- **Volání funkcí**: CALL ukládá návratovou adresu; **zásobníkový rámec** (PUSH EBP; MOV EBP,ESP; SUB ESP,n); zásobník roste **dolů**.
- **Konvence volání**: cdecl (uklízí volající, zprava doleva), pascal (uklízí volaný), stdcall.

Assembler a překlad

- **Assembler** (NASM) překládá JSI → strojový kód; **direktivy/pseudoinstrukce** (DB/DW, RESB, SECTION) řídí překlad (nepřekládají se).
- **Dvouprůchodový překlad** (1. průchod tabulka symbolů, 2. generování) — kvůli dopředným skokům; **makra** (textová expanze, rychlejší než funkce, ale větší kód).

Zřetěžené zpracování instrukcí viz [[okruhy/07-princip-cinnosti-pocitace-pipelining]]; aritmetika a IEEE 754 viz [[okruhy/09-reprezentace-cisel-ieee754]]; segmenty paměti viz [[okruhy/38-sprava-souboru-pameti]].

Související syntéza

- [[synthesis/zasobnik-napric-obory|Zásobník napříč obory]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Volání funkcí □ [[#Symbolický jazyk a instrukce]] - *CALL/RET, co na zásobník?* □ CALL uloží návratovou adresu; na zásobník i EBP, parametry, lokální proměnné. - *Zásobníkový rámeček?* □ PUSH EBP; MOV EBP,ESP; SUB ESP,n; EBP = referenční bod, ESP = vrchol; zásobník roste dolů. - *Konvence volání?* □ cdecl (uklízí volající), pascal/stdcall (uklízí volaný); pořadí parametrů.

Příznaky a skoky □ [[#Symbolický jazyk a instrukce]] - *CF / ZF / OF?* □ carry (bezznaménkové přetečení), zero (výsledek 0), overflow (znaménkové přetečení) — řídí podmíněné skoky. - *Cykly v assembleru?* □ skokovými instrukcemi (JMP, Jcc, LOOP).

Jazyky a překlad □ [[#Assembler a překlad]] - *Symbolický vs. strojový jazyk?* □ Symbolický = čitelné zkratky (~1:1 mapování), strojový = binární; assembler překládá mezi nimi. - *Proč dvouprůchodový překlad?* □ Kvůli dopředným skokům (nejdřív tabulka symbolů).

Činnost počítače □ [[#Činnost počítače]] - *Fetch-decode-execute?* □ Načtení instrukce (IR←[IP]), dekódování, vykonání; IP ukazuje na další instrukci.

Plné znění (ke studiu)

Jazyk symbolických instrukcí je nízkoúrovňový programovací jazyk, který obsahuje **symbolické reprezentace strojových instrukcí**. Je definován výrobcem daného hardware, je založen na zkratkách jmen instrukcí (např. compare -> CMP).

Činnost počítače

[[media/szz-41/media/image12.png]]

Hlavními částmi počítače jsou **CPU** (skládající se z aritmeticko-logické jednotky (**ALU**), **registrů, řadiče a zdroj hodin**), **paměť** a **vstupně-výstupní zařízení**, všechny tyto komponenty jsou propojeny **sběrnici**. Významnými registry pro základní činnost počítače jsou **akumulátor** (střadač) pro výpočty, **instruction register** (IR) obsahující současnou instrukci a **instruction pointer** register (IP pro 16-bit a EIP pro 32-bit) ukazující na následující instrukci v paměti (která bude načtena a vykonána po současné instrukci). Paměť tak současně obsahuje:

- **data**, se kterými se pracuje,
- **instrukce**, které tuto práci (výpočet) řídí.

Princip činnosti PC

Princip činnosti počítače je následující (opakuje se v cyklu):

1. **fetch**: do **IR** se uloží obsah paměti, na který ukazuje **IP**,
2. aktualizuje se **IP** (na následující instrukci - přičte se počet bytů odpovídající provedené instrukci),
3. **decode**: dekóduje se IR, určí se operace a operandy,
4. **execute**: provede se daná instrukce (skoky ještě aktualizují **IP**).

V případě přerušení je obsah některých registrů (instruction pointer, flags, ...) nahrán na zásobník a po obsluze přerušení je stav těchto registrů ze zásobníků obnoven.

Strojový kód

Kód specifický pro daný počítač, **binární jedničky a nuly**, není téměř vůbec přenositelný, v dnešní době nemožné v něm programovat. Výhodou je vysoká efektivita, nevýhodou **nepřenositelnost, nečitelnost pro člověka a složitost psaní**. Právě z těchto důvodů se zavádí symbolický jazyk, který

je čitelnější, ovšem stále má víceméně 1:1 mapování na strojový kód (tedy výhodu efektivity) a poté dále vyšší programovací jazyky, které díky překladačům zajišťují přenositelnost.

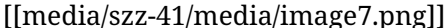
Symbolický jazyk

Také jazyk symbolických instrukcí.

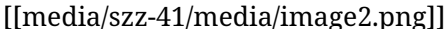
Instrukce

Instrukce jsou **příkazy pro procesor**, obsahují jednoznačný **operační kód** a operandy, příp. adresy operandů. Některé instrukce mohou mít ve své specifikaci napevno nastavené, s kterými registry pracují. Základními typy instrukcí jsou:

- **přenosové:** MOV, PUSH, POP,
- **aritmetické:** ADD, SUB, INC,
 - MUL - pozor, $32b \cdot 32b = 64b$, využívá se EDX:EAX pro výsledek; IMUL - znaménkové násobení,
 - DIV - umí zase dělit 64 bit číslo z EDX:EAX v případě 32-bit musíme rozšířit znaménko do EDX, jinak to nebude fungovat, výsledek je v EAX, v EDX je pak modulo - zbytek, IDIV - znaménkové dělení.
- **posuvy a rotace:** SHL, ROL, ... 

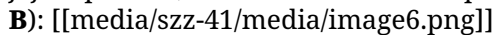


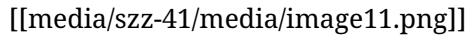
- **logické:** AND, OR, NOT, TEST, XOR, ...
- **skokové:** JMP, JA, LOOP - skáče podle hodnoty counteru ECX - rozdílné od nuly znamená skoč, jinak ne. Skoky mohou prováděny **podmíněně** dle příznaků (např. přetečení).
 - skoky pro **znaménkovou** aritmetiku
 - skoky pro **neznaménkovou** aritmetiku 

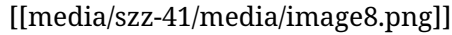


- **řetězové:** umožňují jednodušší a rychlejší práci s poli (automaticky **inkrementují/dekrementují** hodnotu ukazatelů v ESI a EDI). Lze je kombinovat s prefixy REP* pro opakování dle ECX.
 - MOVS(B/W/D) - přesun hodnot odkazovaných z ESI do EDI (B po bytech, W po slovech a D po 4 bytech),
 - CMPS(B/W/D) - porovnání hodnot odkazovaných ESI a EDI,
 - SCAS(B/W/D) - porovnání hodnoty EAX s hodnotou odkazovanou EDI, lze využít pro hledání prvku v poli,
 - LODS(B/W/D) - načtení z adresy odkazované ESI do akumulátoru,
 - STOS(B/W/D) - uložení akumulátoru (AL/AX/EAX) na adresu odkazovanou z EDI.
- **řídící:** CALL, RET (umožňuje odebrat **n** slabik ze zásobníku)
 - CALL je skok, co uloží na zásobník **návratovou adresu** (RET si ji pak vezme a vrátí řízení za instrukci CALL)
 - Parametry se předávají v **globálních proměnných, registrech** nebo na **zásobníku** (dle konvence volání), či kombinací.

- **Zásobníkový rámec** je typická konstrukce ve volání funkcí, tvoří se “záložka” pro jednoduché vyčištění např. lokálních proměnných ze zásobníku (vyčištění znamená změnu hodnoty ukazatele na vrchol zásobníku v registru **ESP**, uložená data tam zůstávají, dokud nejsou přepsána dalším použitím zásobníku - nikdy tato data ale už nelze použít z důvodu možného vzniku přerušení). Register **EBP** pak funguje jako reference, abychom věděli, jak moc věcí jsme už na zásobník zapsali. Při vstupu do funkce **uložíme starou referenci** (cizí referenci toho, kdo funkci volal) z **EBP** na zásobník a vytvoříme si novou referenci - aktuální vrchol zásobníku **ESP**. Místo, které budeme na proměnné potřebovat, vyhradíme odečtením počtu bytů od **ESP**, čímž se posune vrchol zásobníku a vytvoří se požadované místo (odečítá se, protože zásobník je vzhůru nohama a jeho dno je na maximální adrese; roste směrem k adrese 0). Před výstupem z funkce pak musíme obnovit původní obsah registrů **ESP** a **EBP**.

Obecně mají instrukce **proměnnou délku** (ta je např. dána způsobem adresování operandů nebo jejich počtem). Formát instrukce na procesoru intel Pentium je takový (délka může být od **1 B** do **16 B**): 





Konvence volání

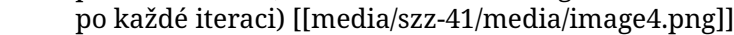
- **pascal** - parametry **uklízí volaný** (tj. uvnitř funkce), parametry se dávají na zásobník **zleva doprava**, tzn. na vrcholu zásobníku je při předání řízení do funkce hodnota **posledního parametru** (využito v Pascalu),
- **cdecl** - parametry **uklízí volající** (tj. až po návratu za funkcí), předávají se **zprava doleva**, tzn. na vrcholu zásobníku je po předání řízení do funkce **první parametr** (jazyk C),
- **stdcall** - parametry **uklízí volaný**, předávají se **zprava doleva** (winapi)

Předávání argumentů **zprava doleva**, tzn. na vrcholu zásobníku je po předání řízení do funkce **první parametr**, umožňuje funkce s proměnným počtem parametrů – první parametr specifikuje jejich počet.

Registry

Máme mnoho registrů pro různé účely různých velikostí. Základní datové registry (například střadač), mají více možností, jak se odkazovat na jejich část, např. **EAX** = celý 32b registr, **AX** = pouze spodních 16b, **AH** = vrchní 8b **AX**, **AL** = spodních 8b **AX**

Základními registry jsou: **EAX**, **EBX**, **ECX**, **EDX**, pro práci se zásobníkem se využívá **ESP** (stack pointer), **EBP** (base pointer), řetězové instrukce využívají **ESI** (zdrojová data) a **EDI** (cílová data). Speciálním je registr **EFLAGS** obsahující příznaky, které se nastavují dle operací: 

- **OF** (overflow flag) = 1 při přetečení znaménkové operace, jinak 0,
- **CF** (carry flag) = 1 při přetečení bezznaménkové operace, jinak 0,
- **SF** (sign flag) = 1 při záporném výsledku
- **ZF** (zero flag) = 1 když je výsledek 0
- **DF** (direction flag) = určuje směr řetězových instrukcí - směr průchodu polem (nastavujeme instrukcemi **CLD** - clear direction flag - nastaví na 0, což znamená zvyšování ukazatele na data po každé iteraci, **STD** - set direction flag - nastaví na 1, což způsobí snižování ukazatele na data po každé iteraci) 

Skoky

- **Short** – **2-bytová** instrukce, která dovoluje skočit na místo v rozsahu +127 and -128 bytů od místa, které následuje za příkazem JMP.
- **Near** – **3-bytová** instrukce, která dovoluje skočit v rozsahu +/- 32KB od následující instrukce v rámci běžného kódového segmentu.
- **Far** – **5-bytová** instrukce, která dovoluje skočit na jakékoliv místo v celém adresovém prostoru.

Paměť

V základním režimu (16b) má fyzická adresa 20b, tedy pro adresaci pomocí registrů chybí 4b. K řešení tohoto problému se vytváří **segmenty** (úseky paměti o velikosti 64 KiB, které lze adresovat 16 bity), každý segment začíná na adrese, která je násobkem 16 (je **posunutá** o 4 bity doleva). Fyzická adresa je pak kombinací **segment:offset** (RegReg:Offset), kdy **fyzická adresa = 16 * segment + offset**.
[[media/szz-41/media/image9.png]]

Little a Big Endian

FPU

[[media/szz-41/media/image3.png]]

Dosud tato otázka řešila pouze operace v ALU, tj. integer a unsigned aritmetika, pro operace s floating point (IEEE754 viz jiné otázky) se používají specializované jednotky (koprocesory), například **FPU** (floating point unit).

Registry FPU pracují v **zásobníkovém režimu**, vršek zásobníku je ST0. Instrukce jsou prefixovány písmenem F, například **FADD**, **FABS**, **FSIN** a mají mnoho možností operandů (implicitní operandy, explicitní operandy, popnutí zásobníku registrů atd.). Např. **FADD st1** provede **st0 = st0 + st1**.

Assembler

Assembler je překladač jazyka **symbolických instrukcí do strojového kódu**, příkladem je **NASM**. Aby mohl být proveden překlad, potřebuje nejen samotné instrukce, ale i dodatečné informace, typicky v podobě **pseudoinstrukcí** nebo **direktiv** (direktiva je příkazem pro překladač, nikoliv instrukcí pro procesor a nejsou překládány do strojového kódu). Jedná se především o:

- Definování konstant (DB, DW, DD, DQ).
- Definování místa v paměti pro uložení dat (RESB, RESW, RESQ).
- Vytváření segmentů paměti (.data, .txt).
- Vkládání jiných souborů (INCLUDE).

Pseudoinstrukce a direktivy tak popisují strukturu paměti, například kde **začíná a končí program** (SECTION .text), **definici dat** (inicializovaná - DB, DW, ... a neinicializovaná - RESB, RESW, ...), **definici konstant, definice jmen** (GLOBAL, EXTERN), příp. také mohou být podporována makra, která překladač textově expanduje (jako v C) nebo nějaký další pre-processing.

Překladač dělá dvouprůchodový překlad, nejprve sestavuje **tabulku symbolů** (ukládá adresy návestí a jmen), kterou následně při druhém průchodu využije (symbol musí být při druhém průchodu **jednoznačně** definován). Dvojí průchod je nutný například kvůli **dopředným** skokům.

Makra Makro je mechanismus, kterým můžeme nadefinovat posloupnost příkazů, vykonávající určitou činnost a tuto posloupnost příkazů **později vložit na různá místa programu**. Překladač tedy kód obsažený v makru kopíruje na všechna místa jeho použití. Nachází se tedy ve výsledném strojovém kódu opakovaně stejné posloupnosti jedniček a nul. Na rozdíl od funkcí, které jsou definované pouze na jednom místě a skáče se tam pomocí CALL a RET.

- **výhody:** rychlejší než funkce (nemusí se nikam skákat a vytvářet rámec),
- **nevýhody:** větší program (více kódu, což může mít špatný vliv na výkon, viz stránkování).

Zdroje

- SZZ okruh 41 — studijní materiály FIT BUT (szz-41.docx). Obrázky: media/szz-41/.

Navigace: [[index| • Přehled okruhů]] · □ [[okruhy/40-objektova-orientace|40. Objektová orientace]] · další: [[okruhy/42-sluzby-aplikacni-vrstvy|42. Služby aplikační vrstvy]] □